

ROI 校正・Canonical 正規化・ SaikuPatchCore 実験コード解説ガイド

Python・画像処理・PatchCore 実装を読み解くための基礎知識レポート

本資料の目的は、今回作成した ROI 校正・canonical dataset 生成・正常モード解析・SaikuPatchCore 基準実験コードを、単に実行するだけでなく、**自分で読んで修正できる状態**にすることである。Python の文法、NumPy/OpenCV/PyTorch のデータ構造、円フィッティング、GrabCut, CNN 特徴, memory bank, k-means, FAISS 最近傍探索までを、実コードの処理順に沿って説明する。

対象： KHI / MVTec AD 形式の局所異常検知実験

基盤： SaikuPatchCore / WideResNet50-2 / FAISS

前処理： Dataset-specific ROI Prior Calibration

作成日： 2026 年 8 月 7 日

本資料中のコード構造・関数名は、今回作成した `khi_roi_patchcore_pipeline_20260807` を基に整理したものである。

目次

第 1 章	まず全体像をつかむ	4
1.1	今回のコードは何をするのか	4
1.2	5 つの主要 Python ファイル	5
第 2 章	Python コードを読むための最低限の文法	6
2.1	変数・型・関数	6
2.2	list・tuple・dict	6
2.3	dataclass	7
2.4	class と self	7
2.5	if・for・内包表記	7
2.6	例外処理	7
2.7	argparse と PowerShell 引数	8
第 3 章	画像は NumPy 配列である	9
3.1	OpenCV 画像の shape	9
3.2	mask は 0/255 または bool の 2 次元配列	9
3.3	dtype の意味	9
3.4	reshape・transpose・permute	10
第 4 章	ROI Prior Calibration の中身	11
4.1	なぜ毎画像ゼロから ROI 検出しないのか	11
4.2	Circle モード：点から円を推定する	11
4.3	RANSAC とは何か	12
4.4	Polygon モード	12
4.5	Smart モードと GrabCut	12
4.6	複数校正画像をどう統合するか	12
第 5 章	Canonical ROI Normalization	13
5.1	なぜ直接 256×256 へ Resize しないのか	13

5.2	make_square_transform	13
5.3	画像と mask で補間法を変える	13
5.4	局所円補正で Sobel を使う理由	14
第 6 章	MVTec AD 形式とファイル探索	15
6.1	カテゴリという言葉の意味	15
6.2	Pathlib	15
6.3	日本語 Windows パスと OpenCV	15
第 7 章	正常モード解析を読む	16
7.1	descriptor 関数	16
7.2	StandardScaler	16
7.3	k-means は「真の正常モード」を自動で発見する魔法ではない	16
第 8 章	PyTorch と CNN 特徴を読む	18
8.1	Tensor の基本 shape	18
8.2	ImageNet 正規化	18
8.3	forward hook	18
8.4	AvgPool2d	19
8.5	layer2 と layer3 の特徴を結合する	19
8.6	ROI mask を feature map へ縮小する	19
8.7	no_grad と GPU	19
第 9 章	memory bank とメモリ不足対策	20
9.1	memory bank とは	20
9.2	なぜ chunk 保存するのか	20
9.3	MiniBatchKMeans	20
9.4	random sampling との違い	21
第 10 章	FAISS 最近傍探索	22
10.1	kNN の意味	22
10.2	Flat・IVF・HNSW	22
第 11 章	テスト推論と異常マップ	23
11.1	infer_resolution の処理順	23
11.2	画像スコア	23
11.3	256 と 512 の融合	24
第 12 章	評価指標と出力ファイル	25

12.1	AUROC と AP	25
12.2	metrics.json を読む	25
第 13 章	ファイル別の読み方	26
13.1	khi_roi_common.py	26
13.2	01_roi_calibration_app.py	26
13.3	02_build_canonical_mvtec_dataset.py	26
13.4	03_analyze_normal_modes.py	26
13.5	04_run_saiku_patchcore_baselines.py	26
第 14 章	主要関数リファレンス	28
第 15 章	コードを自力で解説するときのチェック方法	29
15.1	1 関数につき 5 項目だけ書き出す	29
15.2	shape を print する	29
15.3	PowerShell で構文だけ確認する	29
15.4	小規模 subset で確認してから全量へ進む	30
第 16 章	よく遭遇するエラーの読み方	31
第 17 章	学習順序と到達目標	32
17.1	優先して理解する順序	32
17.2	最終的に説明できるべきこと	32
第 18 章	用語集	33
第 19 章	本レポートで参照した実装	34
まとめ		35

1 まず全体像をつかむ

1.1 今回のコードは何をするのか

今回のコードは、一つの巨大なプログラムではなく、処理を段階ごとに分けた構成である。最初に撮影環境ごとの評価領域を少数画像から校正し、次にその ROI を使って幾何形状を保った正方形画像を作る。その後、正常画像の外観分布を解析し、最後に ROI 内の CNN パッチ特徴だけを用いて SaikuPatchCore 型異常検知を実行する。

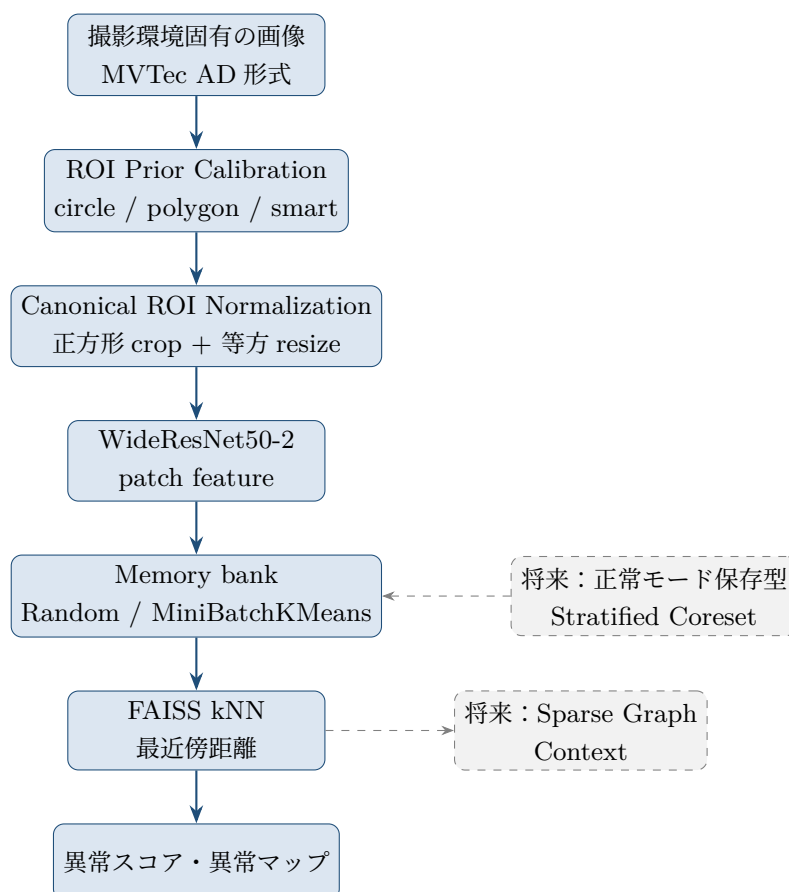


図 1.1: 今回の再実験コードの全体像。灰色は今後追加する研究要素である。

最初に覚えるべきこと

コードを読むときは、関数名を一つずつ暗記するよりも、「この変数は今、画像なのか、mask なのか、特徴ベクトルなのか」を追うことが重要である。今回のコードでは、主に Path, numpy.ndarray, torch.Tensor, dict の 4 種類を追えば流れが見えやすい。

1.2 5つの主要 Python ファイル

表 1.1: 今回の主要ファイルと役割

ファイル	役割
khi_roi_common.py	円 fit, GrabCut, mask 生成, 正方形 crop, padding, resize などの共通関数
01_roi_calibration_app.py	少数の代表正常画像から ROI prior を手動・半自動で作成
02_build_canonical_mvtec_dataset.py	ROI profile を全画像へ適用し, 幾何形状を保った MVTec AD 形式データセットを生成
03_analyze_normal_modes.py	train/good だけから明度・コントラスト・gradient 等を算出し, 正常モード候補を探索
04_run_saiku_patchcore_baselines.py	CNN 特徴抽出, memory bank 作成, k-means 圧縮, FAISS 検索, 異常スコア計算

2 Python コードを読むための最低限の文法

2.1 変数・型・関数

Python では、変数に型名を直接書かなくても値を代入できる。ただし今回のコードでは、読みやすさのために型ヒントを多用している。

```
from pathlib import Path
from typing import List, Optional, Tuple

image_path: Path
points: List[Tuple[int, int]] = []
gt_path: Optional[Path] = None
```

`List[Tuple[int,int]]` は「整数 2 個の組を複数持つリスト」、`Optional[Path]` は「Path または None」という意味である。型ヒントは実行時の変数を固定するものではなく、人間と静的解析ツールに意図を伝える役割が大きい。

2.2 list・tuple・dict

表 2.1: よく使うコンテナ型

型	例	今回の用途
list	[p1,p2,p3]	クリック点, 画像パス, 特徴 chunk 一覧
tuple	(x,y)	座標, shape, 変更しないまとまり
dict	{"cx":10,"r":20}	ROI profile, 設定, metrics
set	{" .png", " .jpg"}	対応拡張子の集合

イメージ: dict は「名前付きの小さな設定表」

ROI profile の `parameters["cx_norm"]` のような書き方は、表の「cx_norm 列」を名前で取り出すイメージである。JSON へ保存すると、Python を終了しても同じ設定を後で復元できる。

2.3 dataclass

今回のコードでは、円や実験設定のように「複数の値をひとまとまりとして扱う」ために `@dataclass` を使っている。

```
@dataclass(frozen=True)
class Circle:
    cx: float
    cy: float
    r: float
```

通常の辞書でも表現できるが、`Circle(cx=..., cy=..., r=...)` と書けるため、何の値かが明確になる。`frozen=True` は、作成後に値を勝手に書き換えにくくする指定である。

2.4 class と self

`CalibrationApp` は、ウィンドウ状態、現在の画像番号、クリック点、確定済み結果を一つのオブジェクトにまとめるクラスである。`self.points` は「このアプリ自身が保持しているクリック点」である。

```
class CalibrationApp:
    def __init__(self, image_paths, mode, output_dir, ...):
        self.image_paths = list(image_paths)
        self.mode = mode
        self.index = 0
        self.points = []
```

関数だけで書くと多数の状態変数を引数で渡し続ける必要があるため、GUI のように状態を保持する処理では `class` が有効である。

2.5 if・for・内包表記

```
images = [
    p for p in folder.iterdir()
    if p.is_file() and p.suffix.lower() in IMAGE_EXTENSIONS
]
```

これは「フォルダ内を順に調べ、条件を満たした `p` だけをリストへ入れる」という `list comprehension` である。同じ処理を通常の `for` 文で書くより短い。

2.6 例外処理

`raise ValueError(...)` は、入力がおかしい状態で処理を続けず、その場で明示的に止めるために使う。

```
if len(points) < 3:
    raise ValueError("circle fit needs at least 3 points")
```


異常検知コードでは、GT が無い、ROI mask が空、CUDA が使えない等を曖昧に続行すると結果を誤解するため、早い段階で停止する方が安全である。

2.7 argparse と PowerShell 引数

argparse は、PowerShell から実験条件を渡す仕組みである。

```
p.add_argument("--output-size", type=int, default=512)
p.add_argument("--refine-circle", action="store_true")
```

--output-size 512 なら整数 512 が入り、--refine-circle は書いたときだけ True になる。コード本文を書き換えず条件を変えられるため、再現可能な実験に適している。

3 画像は NumPy 配列である

3.1 OpenCV 画像の shape

OpenCV で読み込んだカラー画像は、多くの場合 `numpy.ndarray` であり、`shape` は

$$(H, W, C)$$

である。ここで H は高さ、 W は幅、 $C = 3$ は色チャンネルである。OpenCV の色順は基本的に BGR である。

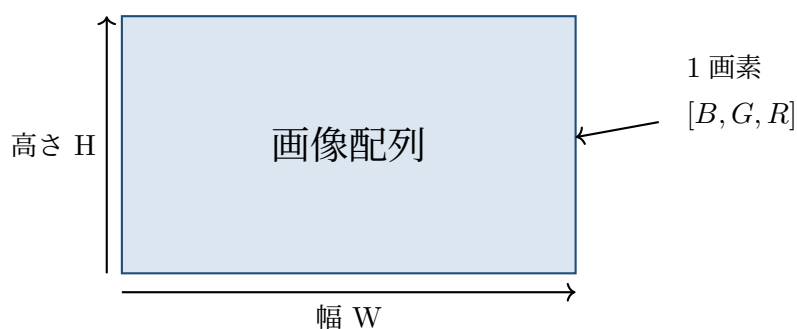


図 3.1: OpenCV カラー画像の基本イメージ。

3.2 mask は 0/255 または bool の 2 次元配列

ROI mask は、評価対象なら 255、対象外なら 0 という 2 次元画像として保存する。計算時には `mask > 127` として bool 配列へ変換する。

```
valid = roi_mask > 127
values = gray[valid]
```

この `gray[valid]` は、2 次元画像から ROI 内部の画素だけを 1 次元に取り出す Boolean indexing である。正常モード解析の平均明度などはこの方式で計算している。

3.3 dtype の意味

画像では `uint8` (0-255) が多く、CNN 特徴や距離計算では `float32` が多い。

表 3.1: 今回よく現れる dtype

dtype	値の例	用途
uint8	0-255	JPEG/PNG 画像, mask
bool	True/False	ROI 内外の選択
float32	実数	CNN 特徴, gradient, FAISS 入力
float64	高精度実数	円 fit や統計計算の一部

3.4 reshape・transpose・permute

機械学習コードでは同じ数値を「どの軸を先に置くか」で形を変える。NumPy は `reshape` や `transpose`, PyTorch は `view`, `reshape`, `permute` を使う。

例えば CNN 特徴は PyTorch では

$$(B, C, H, W)$$

であるが、各 patch を 1 行のベクトルとして扱いたい場合は

$$(B, H, W, C)$$

へ `permute` してから ROI 位置を選択する。

```
features = embedding.permute(0, 2, 3, 1).contiguous()
rows = features[valid[:, 0]]
```

4 ROI Prior Calibration の中身

4.1 なぜ毎画像ゼロから ROI 検出しないのか

同一撮影条件では製品の相対位置がほぼ一定であるという前提を利用し，少数画像で ROI を校正する．これにより，画像全体から毎回物体検出を行うよりも処理が軽く，検査対象を人間が説明しやすい．KHI では circle を使えるが，非円形データでは polygon または smart mask を使う．

4.2 Circle モード：点から円を推定する

ユーザーが円周上に複数点をクリックする．理想的な円周点 (x_i, y_i) は

$$(x_i - c_x)^2 + (y_i - c_y)^2 = r^2$$

を満たす．実際のクリックには誤差があるため，すべての点へ完全一致させるのではなく，RANSAC で外れ点を除き，残った点に最小二乗 fit を行う．

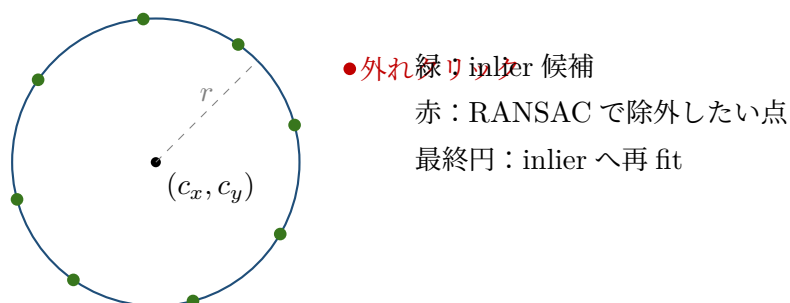


図 4.1: Circle モードの考え方.

4.2.1 最小二乗法をコードでどう書いているか

円の式を展開すると，未知数 c_x, c_y, c について線形に整理できる．コードでは

$$A = \begin{bmatrix} 2x_1 & 2y_1 & 1 \\ \vdots & \vdots & \vdots \\ 2x_n & 2y_n & 1 \end{bmatrix}, \quad b = \begin{bmatrix} x_1^2 + y_1^2 \\ \vdots \\ x_n^2 + y_n^2 \end{bmatrix}$$

を作り，`np.linalg.lstsq(A,b)` で解いている．

lstsq のイメージ

点が多いと方程式の本数が未知数より多くなるため、すべてへ完全一致する解は通常存在しない。
lstsq は「全体として誤差が小さくなる解」を返す。

4.3 RANSAC とは何か

RANSAC では、点群からランダムに 3 点を選び、その 3 点から仮の円を作る。各点はその円から何 pixel ずれているかを調べ、閾値以内を inlier とする。これを何度も繰り返し、最も多くの inlier を説明できた円を採用する。今回の関数 `fit_circle_ransac` は、最後に inlier だけで最小二乗 fit をやり直す。

4.4 Polygon モード

Polygon ではユーザーが頂点を順番にクリックし、`cv2.fillPoly` で内部を 255 に塗る。最終的に PatchCore へ必要なのは「円か多角形か」という名前ではなく、**ROI mask** であるため、以後の処理を共通化できる。

4.5 Smart モードと GrabCut

Smart モードでは、ユーザーが対象領域の外周を大まかになぞる。その粗い polygon を使って、外側を背景、内側を「前景らしい」、さらに内側を少し erode した領域を「確実な前景」として GrabCut へ渡す。

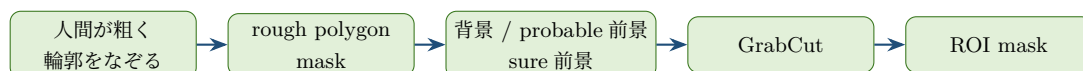


図 4.2: Smart Brush から ROI mask を得る流れ。

4.6 複数校正画像をどう統合するか

Circle では、各画像で得た c_x, c_y, r を画像サイズで正規化し、中央値を代表値とする。ばらつきは MAD (median absolute deviation) で保存する。

$$\text{MAD}(x) = \text{median}(|x_i - \text{median}(x)|)$$

Polygon/Smart では、各画像の mask を共通解像度へ合わせて平均し、ROI 存在確率 map を作る。例えば閾値 0.6 なら、校正画像の 60% 以上で ROI だった画素を代表 ROI とする。

5 Canonical ROI Normalization

5.1 なぜ直接 256×256 へ Resize しないのか

元画像が 1600×1200 の場合、直接 256×256 へ変換すると x 方向と y 方向の縮尺が異なり、円が楕円へ変形する。今回のコードでは、まず ROI を包含する**正方形 crop**を作り、その後に正方形から正方形へ resize する。

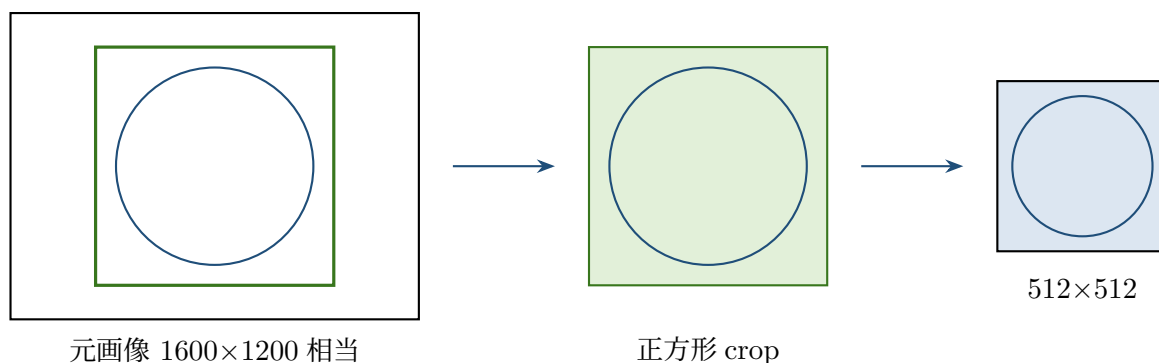


図 5.1: 幾何形状を保つ canonical 化。正方形→正方形なので x/y の縮尺が等しい。

5.2 make_square_transform

ROI mask の非 0 領域から bounding box を求め、その幅と高さの大きい方を基準に正方形を作る。margin を m とすると概念的に

$$L = \max(w_{ROI}, h_{ROI})(1 + 2m)$$

で一辺 L を決める。画像外へはみ出す場合は padding 量を記録する。

5.3 画像と mask で補間法を変える

画像には `cv2.INTER_AREA`、ROI mask と GT には `cv2.INTER_NEAREST` を使用する。mask で線形補間を使うと 0 と 255 の間の中間値が生じ、クラス境界が曖昧になるためである。

表 5.1: 補間方法を分ける理由

対象	補間	理由
通常画像	INTER_AREA	縮小時に画質を保ちやすい
ROI mask	INTER_NEAREST	0/255 のラベルを壊さない
GT mask	INTER_NEAREST	異常ラベル境界を連続値にしない

5.4 局所円補正で Sobel を使う理由

オプションの`--refine-circle`では、保存済み円の周囲だけを探索する。まず Gaussian blur 後に Sobel フィルタで x/y 方向 gradient を計算し、gradient magnitude を作る。候補円周 720 点の gradient 平均が大きい円を採用する。

$$G = \sqrt{G_x^2 + G_y^2}$$

完全な Hough 探索ではなく、 (c_x, c_y, r) の近傍だけを探索するため、撮影環境ごとの ROI prior を活かした軽量の補正である。

6 MVTec AD 形式とファイル探索

6.1 カテゴリという言葉の意味

今回のコードにおける category は、必ずしも意味的な製品クラスを表すだけでなく、train/good と test を持つ MVTec AD 形式フォルダを選択するための単位である。KHI では chip をカテゴリとして使える。

```
category/  
  train/good/  
  test/good/  
  test/chip/  
  ground_truth/chip/  
  roi_masks/train/good/  
  roi_masks/test/good/  
  roi_masks/test/chip/
```

6.2 Pathlib

Path は Windows パスを文字列連結するより安全に扱いやすい。

```
roi_path = path / "roi_masks" / "train" / "good" / filename  
if roi_path.exists():  
    ...
```

/演算子でフォルダを接続でき、.stem で拡張子を除いた名前、.suffix で拡張子を取得する。

6.3 日本語 Windows パスと OpenCV

imread_unicode では、np.fromfile でファイルをバイト列として読み、cv2.imdecode で画像化する。保存時は逆に cv2.imencode して tofile する。これは日本語を含む Windows パスで OpenCV 標準入出力が問題になる場合を避けるための実装である。

7 正常モード解析を読む

7.1 descriptor 関数

`03_analyze_normal_modes.py` では, `train/good` だけを使う. ROI 内画素について平均, 標準偏差, percentile, IQR, gradient 等を計算する. 異常画像や GT を使わないことが重要である.

表 7.1: 正常モード探索用記述量

特徴	直感的な意味
<code>mean_intensity</code>	ROI 全体の明るさ
<code>std_intensity</code>	明るさのばらつき
<code>p05, p50, p95</code>	暗部・中央値・明部の代表値
<code>dynamic_p95_p05</code>	明暗差の大きさ
<code>iqr</code>	中央 50% のばらつき
<code>mean_gradient</code>	エッジ・反射変化の平均的強さ
<code>p95_gradient</code>	強いエッジの上位側
<code>roi_ratio</code>	画像内で ROI が占める割合

7.2 StandardScaler

k-means は距離を使うため, 値の単位が大きい特徴が支配しやすい. そこで各特徴を概ね平均 0, 標準偏差 1 へ変換する.

$$z = \frac{x - \mu}{\sigma}$$

これが `StandardScaler.fit_transform` である.

7.3 k-means は「真の正常モード」を自動で発見する魔法ではない

`KMeans(n_clusters=4)` は, 指定した 4 群へ距離ベースで分割するだけである. したがって, 得られた M00–M03 が研究上意味のある正常モードかどうかは, 代表画像, 時系列, 明度分布, mode 別 FPR

などを確認して判断する必要がある.

8 PyTorch と CNN 特徴を読む

8.1 Tensor の基本 shape

PyTorch の画像 Tensor は通常

$$(B, C, H, W)$$

である。 B は batch 数, C はチャンネル数である。例えば 2 枚の RGB 256×256 画像なら概念的には $(2, 3, 256, 256)$ である。

8.2 ImageNet 正規化

WideResNet50-2 は ImageNet 事前学習重みを使うため, 入力を ImageNet で一般的な平均・標準偏差で正規化する。

```
transforms.Normalize(
    mean=[0.485, 0.456, 0.406],
    std=[0.229, 0.224, 0.225],
)
```

注意：KHI 画像で CNN 重みを学習しているわけではない

今回の標準 PatchCore では WideResNet50-2 の parameter を `requires_grad=False` にし, `eval()` で使用する。KHI の正常画像は CNN を再学習するためではなく, 正常 patch 特徴を memory bank へ蓄えるために使う。

8.3 forward hook

CNN 全体の最終分類結果が欲しいのではなく, 中間層 `layer2[-1]` と `layer3[-1]` の特徴が欲しい。そこで `register_forward_hook` を使う。

```
self.model.layer2[-1].register_forward_hook(self._hook)
self.model.layer3[-1].register_forward_hook(self._hook)
```

forward 時にその層を通ると `self_hook` が呼ばれ, 出力 Tensor を `self.outputs` へ保存する。

イメージ：hook は「途中の測定端子」

CNN を一つの長い配管と考えると、hook は途中の 2 か所に測定器を取り付け、流れてきた特徴を横取りして記録する仕組みである。モデル本体の forward 処理を大きく書き換えずに中間特徴を得られる。

8.4 AvgPool2d

`AvgPool2d(kernel=3, stride=1, padding=1)` は、 3×3 近傍を平均して局所特徴を少し平滑化する。stride=1 なので空間サイズを大きく落とさず、padding=1 で端部サイズを保ちやすい。

8.5 layer2 と layer3 の特徴を結合する

2 つの層は空間解像度が異なるため、`F.unfold` で細かい側の局所領域を展開し、粗い側の特徴とチャネル方向に結合し、`F.fold` で戻す。想定外 shape の場合には nearest interpolation を使う fallback がある。

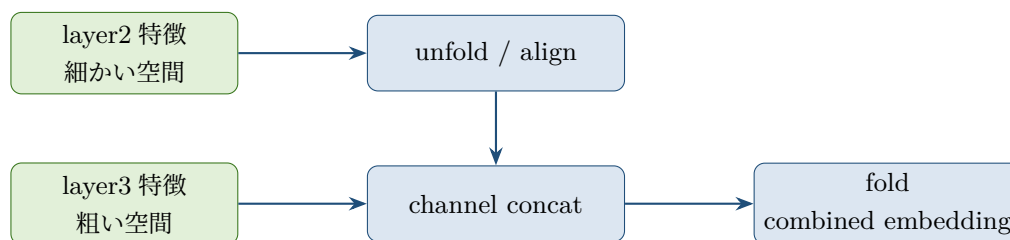


図 8.1: 複数層特徴の結合イメージ。

8.6 ROI mask を feature map へ縮小する

入力画像の ROI mask を CNN 特徴 map の $H \times W$ へ nearest で縮小し、True の位置だけ特徴ベクトルを取り出す。

```
valid = F.interpolate(roi_mask.float(), size=(h, w), mode="nearest") > 0.5
features = embedding.permute(0, 2, 3, 1).contiguous()
rows = features[valid[:, 0]]
```

これにより、背景や padding 部分の patch を memory bank へ入れない。

8.7 no_grad と GPU

`@torch.no_grad()` は勾配計算用情報を保存しない。推論・特徴抽出だけなら backpropagation が不要なので、GPU メモリを減らせる。 `tensor.to(device)` で CPU/GPU を切り替える。

9 memory bank とメモリ不足対策

9.1 memory bank とは

PatchCore では正常画像の各 patch を CNN 特徴ベクトルへ変換し、代表的な正常特徴を保存する。これが memory bank である。テスト patch が memory bank のどの正常特徴にも近くなければ、異常度が高いと考える。

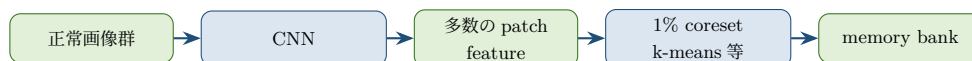


図 9.1: 正常 memory bank 作成の概念。

9.2 なぜ chunk 保存するのか

全画像の全 patch 特徴を Python list に持ち続けると、画像枚数・解像度が増えるほど RAM 使用量が増える。今回のコードでは batch ごとに `chunk_000001.npy` のようにディスクへ保存し、必要なときだけ読み込む。

```

np.save(chunk_path, features)
...
arr = np.load(path, mmap_mode="r")
  
```

`mmap_mode="r"` は、巨大配列を一度に RAM へ全読み込みせず、ファイルをメモリ上の配列のように参照する memory mapping である。

9.3 MiniBatchKMeans

通常 k-means は全データを何度も扱うため大規模 memory bank では重い。MiniBatchKMeans は小さな batch を使って cluster 中心を逐次更新する。今回の `kmeans_stream` では `partial_fit` を繰り返し、1% 程度の prototype へ圧縮する。

1% の意味

元の有効 patch が 100 万個なら、理想的には約 1 万個の代表ベクトルへ減らす考え方である。ただし設定には `max_prototypes` もあり、上限を超えないよう制限する。

9.4 random sampling との違い

Random 1% は代表点をランダムに選ぶ. k-means 1% は特徴空間を cluster に分け, 各 cluster 中心を prototype とする. 今後の研究では, ここを「正常モードごとに最低限の prototype を保証する層化 coreset」へ発展させる.

10 FAISS 最近傍探索

10.1 kNN の意味

テスト patch 特徴 q と memory bank 中の正常特徴 m_j の距離を比較し，最も近い正常特徴との距離を異常度の基礎とする．

$$d(q) = \min_j \|q - m_j\|_2^2$$

コードは互換性のため `n_neighbors=3` で検索するが，異常スコアには `distances[:,0]`，すなわち第 1 近傍距離を用いる．

10.2 Flat・IVF・HNSW

表 10.1: FAISS 検索方式の直感的比較

方式	イメージ	役割
FlatL2	全員と総当たり	遅いが近似誤差のない診断基準
IVF	先に大きな群へ分け， 近そうな群だけ探索	速度と精度の折衷
HNSW	近傍グラフをたどる	高速な近似最近傍候補

IVF では `nlist` が粗い cluster 数，`nprobe` が検索時に何 cluster を見るかを表す．`nprobe` を増やすと一般に検索量が増え，近似誤差を減らしやすいが遅くなる．

11 テスト推論と異常マップ

11.1 infer_resolution の処理順

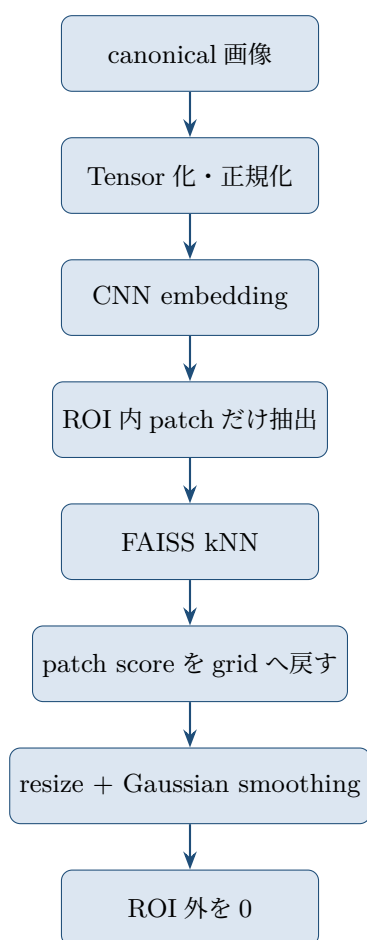


図 11.1: 1 解像度あたりの異常マップ生成.

11.2 画像スコア

各有効 patch の最近傍距離の最大値を画像スコアにする。つまり「画像の中で最も正常 memory bank から離れた場所」を画像全体の異常度としている。

11.3 256 と 512 の融合

low/high 両方を使う場合、画像スコア・異常 map を mean または max で融合する。512 は細部を保持しやすい一方で処理量が増えるため、256-only, 512-only, dual を分けて比較する。

12 評価指標と出力ファイル

12.1 AUROC と AP

Image AUROC は正常画像と異常画像のスコア順位分離を評価する。AP は precision-recall 関係をまとめる指標であり、異常が少ないデータでは AUROC と異なる見え方をする。今回の baseline コードでは Image AUROC/AP を全画像で計算する。

Pixel AUROC/AP はメモリ節約のため ROI 内画素をサンプリングして計算しているため、**最終論文用の厳密 pixel 評価とは区別する必要がある**。AUPRO 等は別評価で追加する。

12.2 metrics.json を読む

表 12.1: 主要 metrics の意味

キー	意味
image_auroc / image_ap	画像単位の識別性能
pixel_auroc_sampled / pixel_ap_sampled	ROI 内サンプル画素による暫定 pixel 性能
vectors_before / vectors_after	memory bank 圧縮前後のベクトル数
compression_ratio_actual	実際に残った割合
memory_bank_mb	prototype 配列の容量
feature_extraction_sec	train 特徴抽出時間
sampler_sec	random / k-means 処理時間
index_build_sec	FAISS index 構築時間
inference_mean_sec	1 画像あたり平均推論時間
inference_p95_sec	遅い側 5% を意識した推論時間
test_total_sec	test 全体の処理時間

13 ファイル別の読み方

13.1 khi_roi_common.py

最初に読むべきファイルである。画像 I/O, 円 fit, mask 生成, canonical crop が独立した小さな関数として並ぶ。以下の順で読むと理解しやすい。

1. Circle, SquareTransform: データ構造を把握する。
2. fit_circle_ransac: 点→円の変換を見る。
3. polygon_mask, grabcut_from_lasso: 任意形状→mask を見る。
4. make_square_transform: mask→正方形領域を見る。
5. canonicalize_image_and_mask: 画像・ROI・GT へ同じ変換を適用する部分を見る。

13.2 01_roi_calibration_app.py

GUI のイベント駆動コードであるため、run() から逆に読むとよい。cv2.waitKey がキー入力を受け、マウス入力は mouse_callback へ入り、Enter 時に estimate_current と confirm_current が呼ばれる。最後に build_profile で複数画像を統合する。

13.3 02_build_canonical_mvtec_dataset.py

main → process_category → process_one の順に追う。1 枚単位の処理は process_one に集約されているため、デバッグ時もここを中心に見るとよい。

13.4 03_analyze_normal_modes.py

descriptor が「1 画像→1 行の数値特徴」を作り、DataFrame へ蓄積する。その後 StandardScaler, KMeans, groupby 集計という典型的な表データ解析の流れである。

13.5 04_run_saiku_patchcore_baselines.py

最も長いので、以下のブロックに分ける。

1. ExperimentConfig / preset

2. MVTec AD データ読み込み
3. FeatureExtractor
4. train 特徴 chunk 保存
5. memory bank sampler
6. FAISS index
7. test inference
8. metrics 保存

いきなり `run_category` 全体を追わず、各ブロックの入力と出力 shape をメモしながら読むことが重要である。

14 主要関数リファレンス

表 14.1: 主要関数と役割

関数	主な入力→出力	役割
<code>list_images</code>	Path → Path list	対応拡張子の画像一覧を作る
<code>imread_unicode</code>	Path → ndarray	日本語パスを考慮した画像読み込み
<code>save_json/load_json</code>	dict ↔ JSON	ROI profile や設定を永続化
<code>fit_circle_least_squares</code>	点群 → Circle	円周点への最小二乗 fit
<code>fit_circle_ransac</code>	点群 → Circle+inlier	外れクリックへ頑健な円推定
<code>polygon_mask</code>	頂点 → mask	多角形 ROI を 0/255 画像化
<code>grabcut_from_lasso</code>	画像 + 粗輪郭 → mask	半自動 foreground 抽出
<code>mask_bbox</code>	mask → x,y,w,h	ROI 外接矩形を求める
<code>make_square_transform</code>	mask → SquareTransform	等方 resize 前の正方形 crop を設計
<code>apply_square_transform</code>	画像 + transform → 画像	crop/padding/resize を実行
<code>canonicalize_image_and_mask</code>	image+ROI+GT → 3 出力	同じ幾何変換を画像・mask へ適用
<code>make_gradient_magnitude</code>	gray → gradient	Sobel による円境界候補強度
<code>refine_circle_local</code>	image+prior → Circle	prior 近傍だけで円を微調整
<code>descriptor</code>	image+ROI → dict	正常モード用の明度・gradient 統計
<code>FeatureExtractor.extract</code>	Tensor → embedding	WRN 中間層特徴を抽出・結合
<code>flatten_valid_features</code>	embedding+ROI → 2D features	ROI 内 patch だけ取り出す
<code>extract_train_chunks</code>	train images → npy 群	特徴を batch ごとにディスク保存
<code>kmeans_stream</code>	chunk 群 → prototypes	MiniBatchKMeans で memory bank 圧縮
<code>build_index</code>	bank → FAISS index	Flat/IVF/HNSW index を構築
<code>infer_resolution</code>	image+ROI+index → score/map	1 解像度の PatchCore 推論
<code>fuse_scores</code>	low/high → fused	256/512 の mean/max 融合
<code>run_category</code>	category → metrics	1 カテゴリの train から test まで統括

15 コードを自力で解読するときのチェック方法

15.1 1 関数につき 5 項目だけ書き出す

未知の関数を読むときは、次の 5 点をノートに書くとよい。

1. 入力変数の型と shape は何か.
2. 出力の型と shape は何か.
3. 何を変更し、何を保存するか.
4. CPU/GPU/RAM/ディスクのどこを多く使うか.
5. 失敗したとき、どの `raise` やエラーが出るか.

15.2 shape を print する

分からないときは、まず shape を表示するのが有効である。

```
print(type(image), image.shape, image.dtype)
print(type(embedding), embedding.shape, embedding.dtype)
print("valid patches =", int(valid.sum()))
```

ただし全 batch で大量に print すると処理が遅くなるため、最初の 1batch だけにするなど制限する。

15.3 PowerShell で構文だけ確認する

コードを変更した後、実験を数時間走らせる前に構文検査する。

```
python -m py_compile .\khi_roi_common.py
python -m py_compile .\01_roi_calibration_app.py
python -m py_compile .\02_build_canonical_mvtec_dataset.py
python -m py_compile .\03_analyze_normal_modes.py
python -m py_compile .\04_run_saiku_patchcore_baselines.py
```

推定実行時間は数秒である。何も表示されず PowerShell へ戻れば、少なくとも Python 構文としては通っている。

15.4 小規模 subset で確認してから全量へ進む

ROI mask が正しいか、Tensor shape が合うか、FAISS index が作れるかは少量データでも確認できる。全量 KHI で初めてエラーを発見すると時間を失うため、数十枚程度の小規模確認→全量の順が安全である。

16 よく遭遇するエラーの読み方

表 16.1: 典型的エラーと確認箇所

エラー例	意味	最初に見る場所
<code>FileNotFoundError</code>	パス/GT/ROI mask が無い	dataset-root, stem, 拡張子
<code>ValueError: ROI mask is empty</code>	ROI 内画素が 0	calibration preview, profile mask
<code>CUDA out of memory</code>	GPU メモリ不足	batch size, 512 処理, 残存 Tensor
<code>MemoryError / allocation error</code>	CPU RAM 不足	uncompressed bank, 巨大 concatenate
<code>FAISS train error</code>	vector 数と nlist 等が不整合	bank shape, nlist, vector 数
<code>shape mismatch</code>	画像/ROI/GT の幾何変換不一致	resize, feature map, interpolate

エラーの最後だけでなく直前の数十行を見る

Python traceback は下から上へ読むと、最後に発生した例外と、その例外までどの関数が呼ばれたかが分かる。研究実験では、エラー直前に出している `vectors=` や `batch=` も原因特定に重要である。

17 学習順序と到達目標

17.1 優先して理解する順序

このコードを解説する目的であれば、Python 全体を体系的に学び終えてから始める必要はない。次の順で学ぶと実コードへ直結する。

1. Path, list, dict, for, if, 関数, class
2. NumPy の shape, slicing, bool mask, dtype
3. OpenCV の画像・BGR・resize・mask・Sobel
4. PyTorch Tensor の BCHW, permute, to(device)
5. CNN 中間特徴, forward hook, pooling
6. k-means と memory bank
7. FAISS kNN と異常スコア
8. AUROC/AP と異常マップ評価

17.2 最終的に説明できるべきこと

本資料を使ってコードを追った後は、少なくとも次を自分の言葉で説明できる状態を目標とする。

- なぜ元画像を直接正方形へ stretch しないのか。
- circle/polygon/smart が最終的に同じ ROI mask へ統一される理由。
- RANSAC と最小二乗がそれぞれ何を担当するか。
- ROI mask を CNN feature map サイズへ縮小する意味。
- WideResNet を KHI 画像で再学習していない理由。
- memory bank が何を保存し、k-means 1% が何を減らすか。
- chunk 保存が RAM 不足へどう効くか。
- FAISS の第 1 近傍距離がなぜ異常スコアになるか。
- AUROC が高くても異常マップ確認が必要な理由。

18 用語集

用語	このコードでの意味
ROI	異常検知の対象とする画像領域
ROI prior	撮影環境ごとに少数画像から記憶した評価領域の事前情報
Canonical image	ROI を基準に正方形 crop し、幾何形状を保って規格サイズへ揃えた画像
Mask	有効領域を 0/255 または False/True で表す 2 次元配列
GT	異常部の正解 mask
Patch	CNN feature map 上の局所位置に対応する特徴単位
Embedding	CNN が画像から生成した特徴ベクトル/特徴 map
Memory bank	正常 patch 特徴の代表集合
Coreset	大量の特徴から情報を保ちながら減らした代表集合
MiniBatchKMeans	小分けデータで cluster 中心を逐次更新する k-means
FAISS	大量ベクトルの最近傍探索ライブラリ
IVF	ベクトルを粗く群分けして探索対象を絞る FAISS index
HNSW	グラフを使った近似最近傍探索方式
AUROC	正常/異常スコアの順位分離能力を表す指標
AP	Precision-Recall 関係を要約する指標
MAD	中央値からの絶対偏差の中央値で、頑健なばらつき指標
RANSAC	外れ値を含む点群から頑健にモデルを推定する反復法
GrabCut	人間の粗い前景/背景情報と画素情報から領域を分割する手法

19 本レポートで参照した実装

本資料は、今回作成した以下の実装内容を直接の対象として整理したものである。一般的な Python・NumPy・OpenCV・PyTorch の説明は、これらの実装を読むために必要な範囲へ限定した。

- khi_roi_common.py
- 01_roi_calibration_app.py
- 02_build_canonical_mvtec_dataset.py
- 03_analyze_normal_modes.py
- 04_run_saiku_patchcore_baselines.py
- README_実験手順.md

注意：研究結果とプログラム解説を区別する

本資料はコード解説用教材であり、RANSAC, GrabCut, k-means, FAISS 等を使ったこと自体を新規研究成果として主張するものではない。研究上の主提案候補である正常モード保存型層化 coreset と Sparse Graph Context は、baseline と正常モード解析の検証後に別段階で評価する。

まとめ

今回のコードを読むための中心概念は、「画像を配列として扱う」→「ROI を mask へ統一する」→「正方形へ幾何正規化する」→「CNN 特徴を patch 単位へ展開する」→「正常特徴の代表集合を作る」→「最近傍距離から異常度を作る」という連鎖である。各関数はこの連鎖のどこか一つを担当している。したがって、関数ごとの入力型・shape・出力を追えば、1000 行を超える実験コードでも処理を分割して理解できる。